

10기_DL팀_신은서_요약본

Tokenization

1. 자연어 처리(NLP)와 토큰화의 필요성

인간의 언어는 컴퓨터가 그대로 이해하기 어렵다.

인간 언어는 크게 세 가지 특성을 가진다.

1. 모호성으로 동일한 표현이 여러 의미로 해석될 수 있음
2. 가변성으로 같은 의미를 다양한 형태로 표현할 수 있음
3. 구조성으로 문법과 규칙이 존재

컴퓨터는 텍스트를 직접 처리할 수 없고, 반드시 숫자 벡터로 변환해야 한다. 이 변환의 첫 번째 단계가 바로 토큰화(Tokenization)이다.

전체 NLP 파이프라인은 "사람 → 텍스트 입력 → 컴퓨터 → 토큰화 → 수치화 → 모델 학습"의 순서로 이루어진다.

2. 토큰화 핵심 개념

토큰화와 관련된 네 가지 핵심 용어를 구분해야 한다.

- **Corpus(말뭉치)**: 분석 대상이 되는 텍스트 데이터셋 전체. 뉴스 기사 모음, 리뷰 데이터셋 등이 해당한다.
- **Token(토큰)**: 토큰화된 결과물인 개별 단위. 단어, 글자, 형태소 등이 토큰이 될 수 있다.
- **Tokenization(토큰화)**: 텍스트를 토큰으로 분리하는 과정 자체를 의미하며, NLP의 첫 번째 단계이다.
- **Tokenizer(토큰라이저)**: 토큰화를 수행하는 규칙 또는 모델. 방식에 따라 결과가 크게 달라진다.

예를 들어 "나는 NLP를 공부한다"를 토큰화하면 ['나는', 'NLP를', '공부한다']로 분리된다.

3. 토큰라이저 구축 방법과 OOV 문제

토큰라이저를 구축하는 방식은 크게 네 가지로 나뉜다.

1. 공백 분할: 띄어쓰기를 기준으로 분리하는 가장 단순한 방식. 한국어, 중국어처럼 띄어쓰기 체계가 복잡한 언어에는 부적합하다.
2. 정규 표현식: 패턴 규칙을 이용해 토큰을 분리. 특수문자, URL 등 구조적 텍스트 처리에 유용하다.
3. 어휘 사전 기반: 사전에 등록된 단어만 토큰으로 인식. 사전에 없는 단어가 들어올 경우 OOV(Out-of-Vocabulary) 문제가 발생할 수 있다.
4. 머신러닝 기반: 데이터에서 토큰화 규칙을 자동으로 학습. BPE, WordPiece 등이 대표적이다.

OOV 문제란 학습 시 어휘 사전에 없던 단어가 입력으로 들어오는 상황을 말한다. 어휘가 커질수록 OOV가 증가하며, 이는 Sparse data와 차원의 저주 문제로 이어진다.

e.g. 어휘 사전이 ['movie', 'good', 'bad']일 때 "masterpiece"가 입력되면 OOV가 발생한다.

이를 해결하기 위해 처음 보는 단어를 이미 아는 조각(subword)으로 분해하는 더 세밀한 토큰화 방식이 필요하다.

4. 토큰화의 세 가지 단위

단어 토큰화

- 띄어쓰기를 기준으로 텍스트를 단어 단위로 분리
- Python의 `split()` 함수가 대표적
- 하지만 'cg'와 'cg.'처럼 구두점이 붙은 경우를 같은 단어로 처리하지 못해 같은 의미인데 다른 토큰이 생성되는 문제가 있으며, 이는 OOV 가능성을 높인다.
- 품사 태깅, 개체명 인식, 기계번역 등에 활용된다.

글자 토큰화

띄어쓰기와 글자 단위를 함께 사용해 분리한다. "현실과 구분" → ['현', '실', '과', '구', '분']처럼 분리된다.

- 장점 : 어휘 사전이 작고 OOV가 감소한다
- 단점 : 글자 단위로 쪼개지므로 의미 정보가 부족해진다
한국어의 경우 자모 토큰화로 초성/중성/종성을 분리해 유사한 발음 패턴을 가진 단어를 처리할 수 있다.
e.g. "현" → ['ㅎ', 'ㄷ', 'ㄴ']으로 분리된다.

형태소 토큰화

- 형태소 : 의미를 가진 최소 단위
 - 자립 형태소
 - 의존 형태소
- 한국어는 교착어이기 때문에 형태소 단위 분석이 특히 중요하며, 검색, 요약, 감성분석 등에 널리 활용된다. "그는 나에게 인사를 했다" → ['그', '는', '나', '에게', '인사', '를', '했', '다']로 분리되며, 품사 태깅 (POS)을 통해 각 형태소에 명사, 조사, 동사 등의 품사 정보를 부착하면 문맥 이해가 가능해진다.

형태소 분석에는 **KoNLPy(Okt, 꼬꼬마)** 및 **NLTK** 라이브러리가 사용된다. Okt는 SNS 텍스트 기반으로 19개 품사를 지원하고, 꼬꼬마는 세종 말뭉치 기반으로 56개 품사를 지원하지만 처리 시간이 더 걸린다. NLTK는 영어를 포함한 다양한 언어를 지원하며 Treebank로 학습된 통계 기반 모델을 사용한다.

하위 단어 토큰화

- 기존 형태소 분석기는 시간이 지남에 따라 등장하는 신조어, 오타자, 은어 등을 처리하기 어렵다는 한계가 있다. 모르는 단어를 적절한 단위로 나누는 것에 취약하고, 신조어가 많아질수록 어휘 사전의 크기가 지나치게 커진다.
e.g. "진짜 멋있네요! 돈썰날만 하네요."를 형태소 분석기로 처리하면 '돈썰날'을 제대로 분리하지 못한다.
- 이를 해결하는 방식이 하위 단어 토큰화로, 하나의 단어를 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화한다.
 - 장점 : 단어 길이 감소, 처리 속도 향상, OOV 문제 완화, 신조어/은어/고유어 문제 완화

- 대표적인 방법으로 BPE, WordPiece, 유니그램 모델이 있다.

5. 하위단어 토큰화 (Subword Tokenization)

BPE (Byte Pair Encoding)

연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복한다. 자주 등장하는 단어는 하나의 토큰으로, 덜 등장하는 단어는 여러 토큰의 조합으로 표현된다. 입력 데이터에서 가장 빈도가 높은 글자 쌍을 탐지해 합치는 과정을 반복한다(예: `abracadabra` → `AracadAra` → `ABcadAB` → `CcadC`).

SentencePiece는 구글이 개발한 오픈소스 BPE 기반 토큰라이저 라이브러리로, 사용자가 직접 하이퍼 파라미터를 설정할 수 있으며 `SentencePieceTrainer.Train()` 으로 학습하고 `SentencePieceProcessor` 로 토큰화를 수행한다.

WordPiece

BPE와 유사하지만 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합한다. $score = f(x,y) / f(x) \cdot f(y)$ 수식을 사용해, 단순히 빈번하게 등장하는 쌍이 아니라 함께 나타날 확률이 높은 쌍을 선택한다.

Tokenizers 라이브러리의 WordPiece API를 사용하면 쉽게 구현 가능하며, 정규화(NFD 유니코드 정규화, 소문자 변환)와 사전 토큰화 기능도 제공한다.

임베딩

6. 임베딩 (Embedding)

텍스트 벡터화와 워드 임베딩

- 텍스트 벡터화 : 텍스트를 숫자로 변환하는 것
초기 방식으로는 원-핫 인코딩(각 단어에 번호를 붙이고 해당 위치만 1, 나머지는 0으로 표현)과 빈도 벡터화가 있으나, 두 방식 모두 벡터의 희소성이 크다는 문제가 있다.
- 워드 임베딩 : 단어를 밀집 벡터(dense vector)로 변환하는 방법. 비슷한 의미의 단어들을 비슷한 벡터 공간에 위치시킨다.
 - Word2Vec, fastText가 대표적이다.
 - 다만 단어를 항상 동일한 값으로 표현하므로 문맥에 따라 의미가 달라지는 단어(동음이의어 등)를 구분하기 어렵다는 한계가 있으며, 이를 해결하기 위해 동적 임베딩이 사용된다.

언어 모델

- 앞에 나온 단어들을 보고 다음에 어떤 단어가 올지 예측하는 모델
- 자기회귀 언어 모델 : 이전 단어들의 정보를 이용해 다음에 나올 단어의 확률을 계산하며, 문장에 이미 등장한 모든 단어를 참고한다.
조건부 확률의 연쇄 법칙 $P(w_1)P(w_2|w_1)\dots P(w_t|w_1,\dots,w_{t-1})$ 을 기반으로 작동한다
- 통계적 언어 모델 : 이전 단어들의 패턴과 빈도를 이용해 다음 단어를 예측하며 주로 마르코프 체인을 사용하지만, 긴 문맥을 이해하거나 미학습 단어를 처리하는 데 한계가 있다.

최근에는 GPT, BERT 같은 대규모 사전 학습 언어 모델이 주류를 이루고 있다.

N-gram

- 연속된 N개의 단어를 묶어 다음 단어를 예측하는 가장 기본적인 언어 모델
- N=1이면 유니그램, N=2면 바이그램, N=3이면 트라이그램
- 이전 N-1개의 단어만 참고해 다음 단어의 확률을 계산하므로 적은 데이터에서도 문장 패턴을 분석할 수 있고 관용 표현 분석에 활용된다.
- Python으로 직접 구현하거나 NLTK의 `nltk.ngrams()` 함수로 쉽게 사용할 수 있다.

TF-IDF (Term Frequency-Inverse Document Frequency)

- 문서에서 중요한 단어를 수치화하는 방법
- 기존의 BoW(Bag-of-Words)가 모든 단어를 동등하게 취급하는 문제를 보완한다.
- TF(단어 빈도) : 특정 단어가 한 문서에 얼마나 자주 등장하는지
- DF(문서 빈도) : 그 단어가 전체 문서 중 몇 개에 등장하는지
- IDF(역문서 빈도) : 여러 문서에서 흔하지 않은 단어일수록 높은 값을 부여함
 - $IDF(t, D) = \log(count(D)/(1 + DF(t, D)))$ 로 계산하며, 분모에 1을 더해 0이 되는 것을 방지하고 로그를 씌워 값이 너무 커지는 것을 방지한다.
- 최종 $TF - IDF = TF \times IDF$ 로 계산되며, 특정 문서에서 자주 등장하면서 다른 문서에서는 잘 나타나지 않는 단어의 중요도를 높게 평가한다.
- 사이킷런의 `TfidfVectorizer` 클래스를 사용해 구현할 수 있으며, 빈도 기반 벡터화는 문장의 순서나 문맥을 고려하지 않고 단어의 의미 자체를 담지 못한다는 한계가 있다.